

Orchestration for Data, Machine Learning and Infrastructure

What is Orchestration?

- Software systems are made of:
 - Processes
 - Workflows
 - Services
 - Systems
- Orchestration ensures they:
 - Run at the right time
 - Run in the right order
 - Work together correctly

What Does Orchestration Do?

- Automates workflows across:
 - Multiple environments
 - Multiple systems
 - Multiple services
- Coordinates complex dependencies
- Manages execution logic:
 - Order
 - Timing
 - Failures
 - Retries

Orchestration in Modern Applications

Orchestration now manages:

- Running applications on a schedule
- Assigning resources on demand
- Rolling back to previous versions
- Managing multi-cloud environments
- Spinning up containers dynamically

Orchestration vs Automation

Important Distinction

- Automation
 - Executes a single task automatically
 - Example: run a script
- Orchestration
 - Coordinates multiple automated tasks
 - Builds an end-to-end workflow
 -



Automation is a subset of orchestration

What Is an Orchestrator?

Definition

An orchestrator:

- Coordinates large workflows
- Manages multiple tasks
- Handles:
 - Dependencies
 - Failures
 - Retries
 - Scheduling
 - Observability

Focus: workflow-level control

Where Is Orchestration Used?

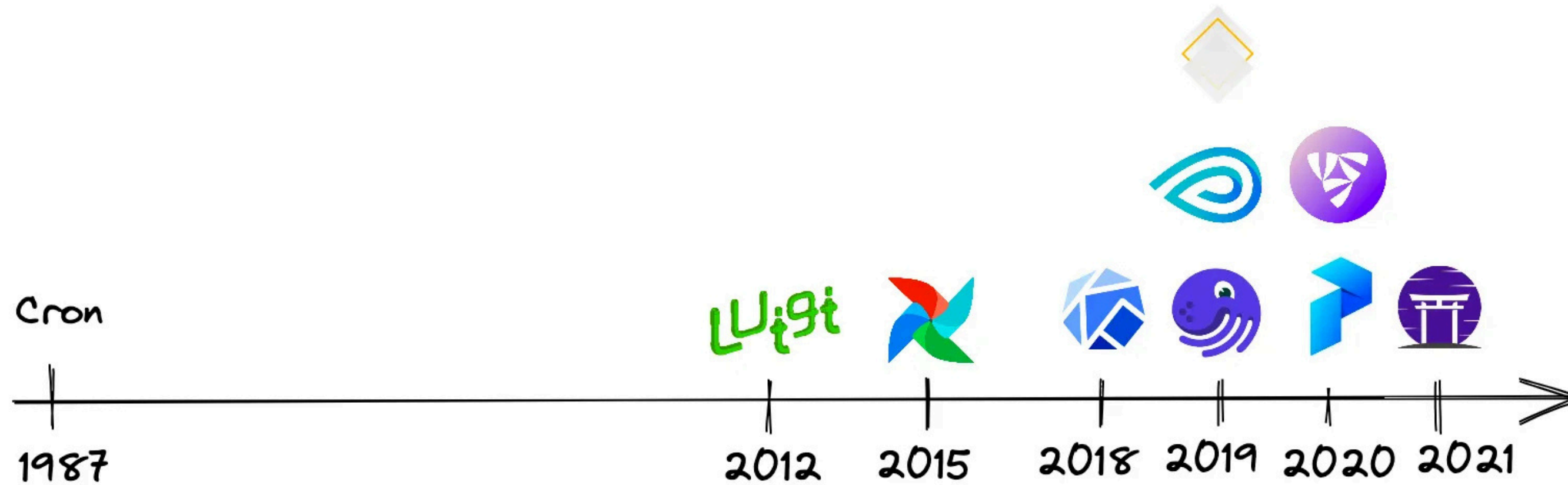
- Cloud orchestration
- Security orchestration
- Container orchestration
- Data & ML orchestration

Each field has specialized tools

Data & ML Orchestrators

- Flyte
- Dagster
- Kubeflow Pipelines
- Prefect
- **Used to:**
 - Build data pipelines
 - Orchestrate ML workflows

History of Orchestration



Data Orchestration

- Ensures data pipelines run:
 - In a specified order
 - In a fault-tolerant manner
 - On a regular cadence if required
- Coordinates all data preparation phases:
 - Ingestion
 - Cleansing
 - Transformation
 - Enrichment
- Goal: reliable and automated data pipelines

Focus: data readiness for analytics & ML

Data Orchestrator

Core Capabilities

- Scheduling
 - Run pipelines regularly to prevent stale data
- Versioning
 - Switch between code versions
 - Roll back to stable versions
- DAGs (Directed Acyclic Graphs)
 - Visualize task and data dependencies
- Parameterization
 - Reuse the same logic with different inputs
- Data lineage
 - Track and visualize how data flows through the pipeline

ML Orchestration

What Is ML Orchestration?

- Orchestrates all phases of the ML lifecycle end to end:
 - Feature engineering
 - Training
 - Evaluation
 - Deployment
 - Monitoring
- Goes beyond data orchestration
- Designed specifically for ML workloads
- Controls the interaction between all ML components

ML Orchestrator

Core Capabilities

- Scheduling
 - Retrain models regularly
 - Essential for online or continuous learning
- Versioning
 - Version data, models, and code
 - Avoid retraining from scratch unnecessarily
- Resource management
 - Control CPUs, GPUs, and memory
 - Ensure scalability
- Dynamic workflows
 - Adapt pipelines based on data or model results
- Caching
 - Skip expensive recomputation when inputs haven't changed

Infrastructure Orchestration

What Is Infrastructure Orchestration?

- Manages compute infrastructure instead of workflows
- Focuses on:
 - Servers
 - Containers
 - Networks
 - Storage
- Automates:
 - Resource allocation
 - Scaling
 - Deployment environments

Examples

- Kubernetes
- Nomad
- Rancher

 Infrastructure orchestration supports data & ML orchestration, but does not replace them

